

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Математическая логика и теория формальных языков
Отчёт о выполнении лабораторной работы №5
«Разработка детерминированного анализатора грамматики
LL(1)»

Вариант 17

Студент,

группа 5130201/30102

_____ Путята М. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Контекстно-свободные грамматики	4
1.2 LL(k)-грамматики	4
1.3 LL(1)-грамматика	5
1.4 Заданная грамматика	5
1.4.1 Множества FIRST и FOLLOW	5
1.4.2 Проверка условия LL(1)	7
1.4.3 Таблица выбора	7
1.4.4 Грамматика простого предшествования	7
1.4.5 Матрица отношений простого предшествования	8
1.5 LL(1)-автомат	9
1.5.1 Примеры вывода дерева цепочки	11
1.6 Семантические действия	11
Список использованной литературы	14

Введение

В ходе выполнения данной лабораторной работы необходимо провести анализ заданной грамматики:

$$S \rightarrow aBS \mid b, \quad A \rightarrow a, \quad B \rightarrow A \mid bSB$$

Для выполнения поставленной задачи необходимо:

1. Определить множества FIRST и FOLLOW для каждого нетерминала грамматики.
2. Построить таблицу выбора.
3. Построить матрицу отношений предшествования.
4. Определить тип грамматики.

1 Математическое описание

1.1 Контекстно-свободные грамматики

Контекстно-свободные грамматики относятся к 2 типу грамматик согласно классификации Хомского. В таких грамматиках productions имеют вид $A \rightarrow \beta$, где A - нетерминал, а β - цепочка, которая может содержать и терминалы и нетерминалы.

Для всех КС-грамматик можно реализовать алгоритм восходящего синтаксического анализа. Восходящий синтаксический анализ заключается в поиске в просматриваемой слева направо сентенциальной форме самой левой связки (подцепочки, совпадающей с правой частью одной из productions, которая заменяется соответствующим нетерминалом в левой части). Данный поиск не прекращается до тех пор, пока не будет получен начальный символ грамматики.

1.2 LL(k)-грамматики

LL(k)-грамматики - класс грамматик, для которых возможен нисходящий синтаксический анализ, в котором:

- входная цепочка просматривается слева направо;
- на каждом шаге анализатор может видеть не более чем на k символов вперёд.

Для построения LL(k)-грамматик необходимо определить следующие множества:

- $FIRST(\alpha)$ - множество терминальных символов, с которых могут начинаться цепочки, выводимые из α . Если $FIRST(\alpha) \cap FIRST(\gamma) = \emptyset$, то можно сделать однозначный выбор между правилами $A \rightarrow \alpha$ и $A \rightarrow \gamma$;
- $FOLLOW(A)$ - множество терминальных цепочек (до k символов), которые могут следовать за A в выводах. Если $FIRST(\alpha) \cap FOLLOW(A) = \emptyset$, можно сделать однозначный выбор между правилами $A \rightarrow \alpha$ и $A \rightarrow \varepsilon$.

Формально, в LL(k)-грамматике, если:

- $S \rightarrow^* wA\alpha \rightarrow w\beta\alpha \rightarrow^* wx$
- $S \rightarrow^* wA\alpha \rightarrow w\gamma\alpha \rightarrow^* wy$

где $x, y \in V_T^*$ выполняется условие:

$$FIRST_k(x) = FIRST_k(y) \rightarrow \beta = \gamma$$

Грамматика является LL(k) тогда и только тогда, когда для каждого нетерминала A и двух любых его productions $A \rightarrow \alpha$ и $A \rightarrow \beta$ выполняется:

$$FIRST_k(\alpha FOLLOW_k(A)) \cap FIRST_k(\beta FOLLOW_k(A)) = \emptyset$$

1.3 LL(1)-грамматика

Грамматика является LL(1), если выполняются следующие условия для каждого нетерминала A с продукциями $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$:

- $\forall i, j \in \{1, \dots, n\} : i \neq j \Rightarrow \text{FIRST}_1(\alpha_i) \cap \text{FIRST}_1(\alpha_j) = \emptyset$
- $\forall i, j \in \{1, \dots, n\} : (i \neq j) \wedge (\alpha_i \rightarrow^* \varepsilon) \Rightarrow \text{FIRST}_1(\alpha_j) \cap \text{FOLLOW}_1(A) = \emptyset$

Функции FIRST и FOLLOW для LL(1)

- $\text{FIRST}_1(\alpha) = \{x \in V_T : \alpha \rightarrow^* x\beta\} \cup \{\varepsilon : \alpha \rightarrow^* \varepsilon\}$
- $\text{FOLLOW}_1(A) = \{x \in V_T : S \rightarrow^* \beta Ax\gamma\} \cup \{\$: S \rightarrow^* \beta A\}$

LL(1)-анализатор

LL(1)-анализатор — это нисходящий детерминированный распознаватель, строящий левый вывод цепочки. При принятии решения он использует текущий нетерминал на вершине стека и один символ текущего входа. Его работа основана на таблице выбора продукций, строки которой соответствуют нетерминалам, а столбцы — терминалам и маркеру конца \$.

Построение таблицы выбора начинается с расширенной грамматики с дополнительным правилом $S' \rightarrow S \$$ и использует множества FIRST и FOLLOW. Для каждой продукции $A \rightarrow \alpha$ ячейка $M[A, t]$ заполняется следующим образом:

- если $t \in \text{FIRST}(\alpha)$, то в $M[A, t]$ записывается $A \rightarrow \alpha$;
- если $\varepsilon \in \text{FIRST}(\alpha)$ и $t \in \text{FOLLOW}(A)$, то в $M[A, t]$ также записывается $A \rightarrow \alpha$;
- все незаполненные ячейки означают ошибку.

1.4 Заданная грамматика

Продукции заданной грамматики:

$$S \rightarrow aBS \mid b$$

$$A \rightarrow a$$

$$B \rightarrow A \mid bSB$$

1.4.1 Множества FIRST и FOLLOW

Если для двух альтернатив $A \rightarrow \alpha$ и $A \rightarrow \beta$ множества $\text{FIRST}(\alpha)$ и $\text{FIRST}(\beta)$ не пересекаются, то можно однозначно выбрать нужное правило. Если $\varepsilon \in \text{FIRST}(\alpha)$, то необходимо, чтобы $\text{FOLLOW}(A)$ не пересекалось с $\text{FIRST}(\beta)$ также для однозначного выбора между альтернативами $A \rightarrow \alpha$ и $A \rightarrow \varepsilon$

Вычисление FIRST:

1. Для каждого терминала x : $\text{FIRST}(x) = \{x\}$.
2. Для нетерминалов множества изначально пусты и к ним последовательно добавляются элементы множеств FIRST правых частей правил вывода.

Вычисление FOLLOW:

1. в $\text{FOLLOW}(S')$ заносится «\$».
2. Для каждого правила вывода $A \rightarrow \alpha B \beta$ в $\text{FOLLOW}(B)$ добавляются элементы из $\text{FIRST}(\beta)$. Если цепочка β может быть пустой, то также добавляется $\text{FOLLOW}(A)$.

Построение множеств FIRST :

- $\text{FIRST}(a) = \{a\}$;
- $\text{FIRST}(b) = \{b\}$;
- $\text{FIRST}(S) = \text{FIRST}(aBS) \cup \text{FIRST}(b) = \{a\} \cup \{b\} = \{a, b\}$;
- $\text{FIRST}(S') = \text{FIRST}(S\$) = \text{FIRST}(S) = \{a, b\}$;
- $\text{FIRST}(A) = \text{FIRST}(a) = \{a\}$;
- $\text{FIRST}(B) = \text{FIRST}(A) \cup \text{FIRST}(bSB) = \{a\} \cup \{b\} = \{a, b\}$

Построение множеств FOLLOW:

- $\text{FOLLOW}(S') = \{\$ \}$
- $\text{FOLLOW}(S) \supset \{\$ \}$ (из $S' \rightarrow S \$$)
- $\text{FOLLOW}(S) \supset \text{FIRST}(B) = \{a, b\}$ (из $B \rightarrow bSB$)
- $\text{FOLLOW}(B) \supset \text{FIRST}(S) = \{a, b\}$ (из $S \rightarrow aBS$)
- $\text{FOLLOW}(A) \supset \text{FOLLOW}(B) = \{a, b\}$ (из $B \rightarrow A$)

В таблицах 1 представлены множества FIRST и FOLLOW для заданной грамматики.

Таблица 1. Множества FIRST и FOLLOW

Нетерминал	FIRST	FOLLOW
S'	{a, b}	{ \$ }
S	{a, b}	{a, b, \$ }
A	{a}	{a, b}
B	{a, b}	{a, b}

1.4.2 Проверка условия LL(1)

Грамматика будет являться LL(1), если для любых двух альтернатив одного нетерминала $A \rightarrow \alpha$ и $A \rightarrow \beta$ выполняется $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$. Условие $\varepsilon \in \text{FIRST}(\alpha) \Rightarrow \text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$ проверяться не будет, так как в продукциях грамматики отсутствует ε .

- Проверка для S: $\text{FIRST}(aBS) \cap \text{FIRST}(b) = \{a\} \cap \{b\} = \emptyset$;
- Проверка для B: $\text{FIRST}(A) \cap \text{FIRST}(bSB) = \{a\} \cap \{b\} = \emptyset$;
- Нетерминал A имеет одну продукцию - конфликтов нет;

Конфликты не найдены, следовательно грамматика является LL(1).

1.4.3 Таблица выбора

На основе вычисленных множеств FIRST и FOLLOW была построена таблица выбора, на которой отображены соответствующие продукции и выводы ошибок. Таблица выбора представлена в таблице 2.

Таблица 2. таблица выбора

Нетерминал	a	b	\$
S'	$S' \rightarrow S \$$	$S' \rightarrow S \$$	Ошибка
S	$S \rightarrow aBS$	$S \rightarrow b$	Ошибка
A	$A \rightarrow a$	Ошибка	Ошибка
B	$B \rightarrow A$	$B \rightarrow bSB$	Ошибка

1.4.4 Грамматика простого предшествования

Метод отношений предшествования позволяет выделить связку, анализируя отношения между парами соседних символов сентенциальной формы. Вводятся три бинарных отношения:

- $X \prec Y$ - символ X может находиться непосредственно перед левой границей связки, содержащей Y;
- $X \doteq Y$ - оба символа находятся внутри одной связки;
- $X \succ Y$ - символ X может находится непосредственно перед правой границей связки, содержащей Y.

Отношения определяются следующими правилами:

- $X \equiv Y \iff \exists A \rightarrow \alpha XY\beta \in R$;
- $X < Y \iff \exists A \rightarrow \alpha XM\beta \in R, \quad M \rightarrow^+ Y\gamma$
- $X > Y \iff \exists A \rightarrow \alpha MZ\beta \in R, \quad M \rightarrow^+ \gamma X, \quad Z \rightarrow^* Y\delta, \quad Y \in T$.

Грамматика простого предшествования - грамматика, для которой выполняется следующее:

- матрица данных отношений не содержит ε -продукций;
- матрица данных отношений содержит не более одного отношения на пару символов;
- все правые части различны.

Анализ выполняется при помощи стека: связкой считается самая левая подстрока, ограниченная слева символом "<" и справа символом ">" а внутри содержащая только "=". После нахождения связка сворачивается к соответствующему нетерминалу, и процесс продолжается.

Способы нахождения данных отношений

Отношение одновременной сворачиваемости $\equiv$

1. Два символа X и Y могут иметь отношение одновременной сворачиваемости ($X \equiv Y$), если существует правая часть какой-либо продукции грамматики, в которой эти два символа стоят рядом.

Отношение сворачиваемости $<$

1. Находим в правилах все пары рядом стоящих <символ> <нетерминал>.
2. <символ> будет находиться в отношении $<$ со всеми символами, с которых начинаются цепочки, выводимые из <нетерминала>.

Отношение сворачиваемости $>$

1. Находим в правилах все пары рядом стоящих <нетерминал> <символ>.
2. Все символы, которыми кончаются цепочки, выводимые из <нетерминала>, будут находиться в отношении $>$ с теми терминалами, с которых начинаются цепочки, выводимые из <символа>.

1.4.5 Матрица отношений простого предшествования

Заменяем продукцию $S' \rightarrow S \$$ на $S' \rightarrow \# S \#$.

- **Отношения $\equiv$.** Пары: $\#S, S\#, aB, BS, bS, SB$.

- **Отношения " $<$ ".** Пары: $\#S$, aB , BS , bS , SB .
Отсюда следует: $\{\#, B, b\} < \{a, b\}$ и $\{a, S\} < \{a, b, A\}$
- **Отношения " $>$ ".** Пары: $S\#$, BS , SB .
Отсюда следует: $\{S, b\} > \{\#, B, A, a\}$ и $\{B, A, a\} > \{a, b\}$

Для грамматики была построена матрица отношений простого предшествования. В полученной матрице наблюдается много ячеек, в которых находится несколько символов сразу, что указывает на наличие конфликтов. Матрица предшествования представлена в таблице 3.

Таблица 3. Матрица предшествования

	#	S	A	B	a	b
#		$\equiv$			$<$	$<$
S	$\equiv >$		$< >$	$\equiv >$	$< >$	$<$
A		$>$			$>$	$>$
B		$\equiv >$			$< >$	$< >$
a		$>$	$<$	$\equiv$	$< >$	$< >$
b	$>$	$\equiv$	$>$	$>$	$< >$	$<$

Конфликт $S \equiv \#$ и $S > \#$ объясняется тем, что:

- в правой части правила « $S' \rightarrow \#S\#$ » присутствует « $S\#$ »;
- из нетерминала S выводится цепочка оканчивающаяся символом « S », а символ $\#$ начинается с терминала $\#$.

1.5 LL(1)-автомат

LL(1)-анализ выполняется при помощи стекового автомата, управляемого таблицей выбора. В начальный момент в стек помещается маркер конца входа $\$$ и начальный символ S' (на вершине стека).

На каждом шаге анализатор сравнивает вершину стека X с текущим символом входа a :

- если $X = a$ — терминал совпадает с текущим символом входа: снять X со стека и перейти к следующему символу входа;
- если $X = \$$ и $a = \$$ — цепочка принята (accept);

- если X — нетерминал: обратиться к ячейке $M[X, a]$ таблицы выбора; если продукция $X \rightarrow \alpha$ найдена, заменить X на вершине стека цепочкой α , помещая символы в обратном порядке (крайний левый символ оказывается на вершине); если ячейка пуста — зафиксировать ошибку и остановить разбор.

Построенная для расширенной грамматики с дополнительным правилом $S' \rightarrow S\$$ таблица выбора содержит 4 строки и 3 столбца, как показано в таблице 2.

Работа автомата на примере цепочки aab представлена в таблице 4.

Таблица 4. Работа LL(1)-анализатора для цепочки aab

Шаг	Стек	Вход	Действие
1	$S' \$$	$aab\$$	$S' \rightarrow S\$$
2	$S \$ \$$	$aab\$$	$S \rightarrow aBS$
3	$a B S \$ \$$	$aab\$$	совпадение a
4	$B S \$ \$$	$ab\$$	$B \rightarrow A$
5	$A S \$ \$$	$ab\$$	$A \rightarrow a$
6	$a S \$ \$$	$ab\$$	совпадение a
7	$S \$ \$$	$b\$$	$S \rightarrow b$
8	$b \$ \$$	$b\$$	совпадение b
9	$\$ \$$	$\$$	совпадение $\$$
10	$\$$	$\$$	accept

1.5.1 Примеры вывода дерева цепочки

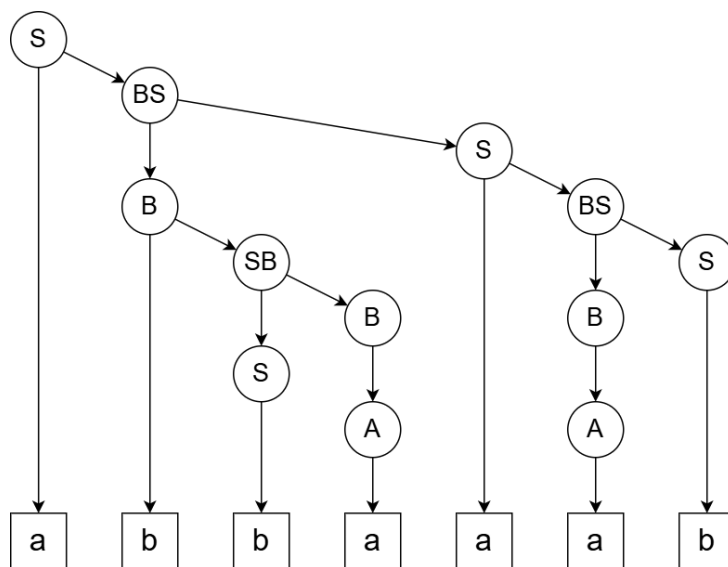


Рис. 1. Пример вывода дерева цепочки №1

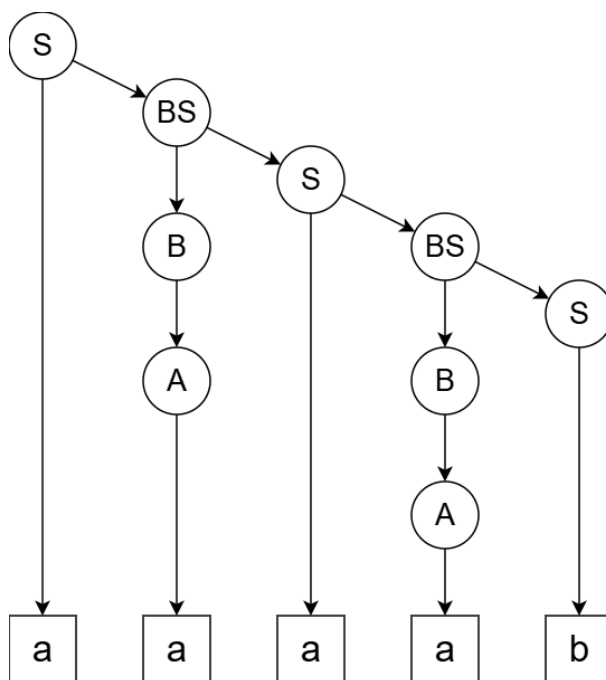


Рис. 2. Пример вывода дерева цепочки №2

1.6 Семантические действия

В работе реализована музыкальная интерпретация: терминал *a* содержит строку «♪», терминал *b* — «♩».

Для каждой продукции задано собственное семантическое действие, генерирующее текстовое описание музыкальной фразы путём конкатенации значений, как показано в таблице 5.

Таблица 5. Семантические действия продукций

Продукция	Семантическое действие
$S' \rightarrow S$	вернуть S
$S \rightarrow aBS$	вернуть $a + B + S$
$S \rightarrow b$	вернуть b
$A \rightarrow a$	вернуть a
$B \rightarrow A$	вернуть A
$B \rightarrow bSB$	вернуть $b + S + B$

Таким образом, результатом разбора цепочки является строка, представляющая собой конкатенацию музыкальных элементов в порядке их появления в выводе. Например, цепочка aab порождает строку «♪♪♪», а цепочка b — строку «♪».

Заключение

В ходе выполнения лабораторной работы был проведён анализ контекстно-свободной грамматики:

$$S \rightarrow aBS \mid b, \quad A \rightarrow a, \quad B \rightarrow A \mid bSB$$

Для каждого нетерминала были вычислены множества FIRST и FOLLOW. Проверка условия LL(1) показала, что для всех нетерминалов с несколькими альтернативами множества FIRST попарно не пересекаются: для S — $\text{FIRST}(aBS) \cap \text{FIRST}(b) = a \cap b = \emptyset$, для B — $\text{FIRST}(A) \cap \text{FIRST}(bSB) = a \cap b = \emptyset$.

Также была построена матрица отношений простого предшествования с целью проверки принадлежности грамматики этому классу. Однако в полученной матрице были обнаружены конфликты: многие пары содержат одновременно несколько отношений, что нарушает требование наличия не более одного отношения на каждую пару символов. Следовательно, грамматика не является грамматикой простого предшествования.

В рассматриваемой грамматике не была найдена левая рекурсия, но были найдены правые рекурсии. После анализа заданной грамматики, можно сделать вывод, что данная грамматика является LL(1)-грамматикой, которая допускает нисходящий синтаксический анализ.

Масштабирование

- создать программную реализацию данной грамматики.

Список использованной литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 21.05.2026).

Таблица 6. Первые 10 строк таблицы вычисления статистики Колмогорова

i	$X_{(i)}$	$\frac{i-1}{n}$	$\frac{i}{n}$	$F_0(X_{(i)})$	$\left F_0 - \frac{i-1}{n}\right $	$\left F_0 - \frac{i}{n}\right $	max
1	-1.493	0.0000	0.0200	0.0861	0.0861	0.0661	0.0861
2	-1.348	0.0200	0.0400	0.2611	0.2411	0.2211	0.2411
3	-1.287	0.0400	0.0600	0.3688	0.3288	0.3088	0.3288
4	-1.266	0.0600	0.0800	0.4090	0.3490	0.3290	0.3490
5	-1.220	0.0800	0.1000	0.5000	0.4200	0.4000	0.4200
6	-1.192	0.1000	0.1200	0.5557	0.4557	0.4357	0.4557
7	-1.189	0.1200	0.1400	0.5616	0.4416	0.4216	0.4416
8	-1.185	0.1400	0.1600	0.5695	0.4295	0.4095	0.4295
9	-1.167	0.1600	0.1800	0.6045	0.4445	0.4245	0.4445
10	-1.158	0.1800	0.2000	0.6217	0.4417	0.4217	0.4417

Таблица 7. Строка с максимальным значением статистики Колмогорова

i	$X_{(i)}$	$\frac{i-1}{n}$	$\frac{i}{n}$	$F_0(X_{(i)})$	$\left F_0 - \frac{i-1}{n}\right $	$\left F_0 - \frac{i}{n}\right $	max
6	-1.192	0.1000	0.1200	0.5557	0.4557	0.4357	0.4557